TUTORIAL I

THE BASICS

I.1. WHAT IS LaTeX?

The short and simple answer is that LaTeX is a typesetting program and is an extension of the original program TeX written by Donald Knuth. But then what is a *typesetting program*?

To answer this, let us look at the various stages in the preparation of a document using computers.

1. The text is *entered* into the computer.
2. The input text is *formatted* into lines, paragraphs and pages.
3. The output text is *displayed* on the computer screen.
4. The final output is *printed*.

In most *word processors* all these operations are integrated into a single application package. But a typesetting program like TeX is concerned only with the second stage above. So to typeset a document using TeX, we type the text of the document and the necessary formatting commands in a *text editor* (such as `Emacs` in GNU/Linux) and then compile it. After that the document can be viewed using a *previewer* or printed using a *printer driver*.

TeX is also a *programming language*, so that by learning this language, people can write code for additional features. In fact LaTeX itself is such a (large) collection of extra features. And the collective effort is continuing, with more and more people writing extra *packages*.

I.1.1. A small example

Let us see LaTeX in action by typesetting a short (really short) document. Start your favorite text editor and type in the lines below *exactly* as shown

```
\documentclass{article}
\begin{document}
This is my \emph{first} document prepared in \LaTeX.
\end{document}
```

Be especially careful with the `\` character (called the *backslash*) and note that this is different from the more familiar `/` (the *slash*) in `and/or` and save the file onto the hard disk as `myfile.tex`. (Instead of `myfile` you can use any name you wish, but be sure to have `.tex` at the end as the *extension*.) The process of compiling this and viewing the output depends on your operating system. We describe below the process of doing this in GNU/Linux.

At the shell prompt type

```
latex myfile
```

You will see a number of lines of text scroll by in the screen and then you get the prompt back. To view the output in screen, you must have the X Window running. So, start `X` if you have not done so, and in a terminal window, type

```
xdvi myfile
```

A window comes up showing the output below

> This is my *first* document prepared in LaTeX.

Now let us take a closer look at the *source file* (that is, the file you have typed). The first line `\documentclass{article}` tells LaTeX that what we want to produce is an article. If you want to write a book, this must be changed to `\documentclass{book}`. The whole document we want to typeset should be included between `\begin{document}` and `\end{document}`. In our example, this is just one line. Now compare this line in the source and the output. The first three words are produced as typed. Then `\emph{first}`, becomes *first* in the output (as you have probably noticed, it is a common practice to *emphasize* words in print using italic letters). Thus `\emph` is a *command* to LaTeX to typeset the text within the braces in *italic*[1]. Again, the next three words come out without any change in the output. Finally, the input `\LaTeX` comes out in the output as LaTeX.

Thus our source is a mixture of text to be typeset and a couple of LaTeX *commands* `\emph` and `\LaTeX`. The first command changes the input text in a certain way and the second one generates new text. Now call up the file again and add one more sentence given below.

```
This is my \emph{first} document prepared in \LaTeX. I typed it
on \today.
```

What do you get in the output? What new text does the command `\today` generate?

1.1.2. Why LaTeX?

So, why all this trouble? Why not simply use a word processor? The answer lies in the motivation behind TeX. Donald Knuth says that his aim in creating TeX is to *beautifully* typeset *technical documents* especially those containing *a lot of Mathematics*. It is very difficult (sometimes even impossible) to produce complex mathematical formulas using a word processor. Again, even for ordinary text, if you want your document to look *really beautiful* then LaTeX is the natural choice.

1.2. SIMPLE TYPESETTING

We have seen that to typeset something in LaTeX, we type in the text to be typeset together with some LaTeX commands. Words must be separated by spaces (does not matter how many) and lines maybe broken arbitrarily.

The end of a paragraph is specified by a *blank line* in the input. In other words, whenever you want to start a new paragraph, just leave a blank line and proceed. For example, the first two paragraphs above were produced by the input

[1] This is not really true. For the real story of the command, see the section on fonts.

```
We have seen that to typeset something in \LaTeX, we type in the
text to be typeset together with some \LaTeX\ commands.
Words must be separated by spaces (does not matter how many)
and lines maybe broken arbitrarily.

The end of a paragraph is specified by a \emph{blank line}
in the input. In other words, whenever you want to start a new
paragraph, just leave a blank line and proceed.
```

Note that the first line of each paragraph starts with an *indentation* from the left margin of the text. If you do not want this indentation, just type `\noindent` at the start of each paragraph for example, in the above input, `\noindent We have seen ...` and `\noindent The end of ...` (come on, try it!) There is an easier way to suppress paragraph indentation for *all* paragraphs of the document in one go, but such tricks can wait.

I.2.1. Spaces

You might have noticed that even though the length of the lines of text we type in a paragraph are different, in the output, all lines are of equal length, aligned perfectly on the right and left. TEX does this by adjusting the space between the words.

In traditional typesetting, a little extra space is added to periods which end sentences and TEX also follows this custom. But how does TEX know whether a period ends a sentence or not? It assumes that every period *not following an upper case letter* ends a sentence. But this does not always work, for there are instances where a sentence does end in an upper case letter. For example, consider the following

> Carrots are good for your eyes, since they contain Vitamin A. Have you ever seen a rabbit wearing glasses?

The right input to produce this is

```
Carrots are good for your eyes, since they contain Vitamin A\@. Have
you ever seen a rabbit wearing glasses?
```

Note the use of the command `\@` *before* the period to produce the extra space after the period. (Remove this from the input and see the difference in the output.)

On the other hand, there are instances where a period following a lowercase letter does not end a sentence. For example

> The numbers 1, 2, 3, etc. are called natural numbers. According to Kronecker, they were made by God; all else being the work of Man.

To produce this (without extra space after `etc.`) the input should be

```
The numbers 1, 2, 3, etc.\ are called natural numbers. According to
Kronecker, they were made by God;all else being the works of Man.
```

Here, we use the command `\ ` (that is, a backslash and a *space*—here and elsewhere, we sometimes use ␣ to denote a space in the input, especially when we draw attention to the space).

There are other situations where the command `\ ` (which always produce a space in the output) is useful. For example, type the following line and compile it.

```
I think \LaTeX is fun.
```

You get

I think LaTeXis fun.

What happened to the *space* you typed between `\LaTeX` and `is`? You see, TEX gobbles up all spaces after a command. To get the required sequence in the output, change the input as

```
I think \LaTeX\ is fun.
```

Again, the command `\ ` comes to the rescue.

1.2.2. Quotes

Have you noticed that in typesetting, opening quotes are different from closing quotes? Look at the TEX output below

Note the difference in right and left quotes in ‘single quotes’ and “double quotes”.

This is produced by the input

```
Note the difference in right and left quotes in ‘single quotes’
and ‘double quotes’’.
```

Modern computer keyboards have a key to type the symbol ` which produces a left quote in TEX. (In our simulated inputs, we show this symbol as ‘.) Also, the key ' (the usual ‘typewriter’ quote key, which also doubles as the apostrophe key) produces a left quote in TEX. Double quotes are produced by typing the corresponding single quote twice. The ‘usual’ double quote key " can also be used to produce a *closing* double quote in TEX.

If your keyboard does not have a left quote key, you can use `\lq` command to produce it. The corresponding command `\rq` produces a right quote. Thus the output above can also be produced by

```
Note the difference in right and left quotes in \lq single
quotes\rq\ and \lq\lq double quotes\rq\rq.
```

(Why the command `\ ` after the first `\rq`?)

1.2.3. Dashes

In text, dashes are used for various purposes and they are distinguished in typesetting by their lengths; thus short dashes are used for hyphens, slightly longer dashes are used to indicate number ranges and still longer dashes used for parenthetical comments. Look at the following TEX output

X-rays are discussed in pages 221–225 of Volume 3—the volume on electromagnetic waves.

This is produced from the input

```
X-rays are discussed in pages 221--225 of Volume 3---the volume on
electromagnetic waves.
```

Note that a single dash character in the input - produces a hyphen in the output, two dashes -- produces a longer dash (–) in the output and three dashes --- produce the longest dash (—) in the output.

1.2.4. Accents

Sometimes, especially when typing foreign words in English, we need to put different types of accents over the letters. The table below shows the accents available in LaTeX. Each column shows some of the accents and the inputs to generate them.

ò	\‘o	ó	\’o	ô	\^o	õ	\~o
ō	\=o	ȯ	\.o	ö	\"o	ç	\c c
ǒ	\u o	ǒ	\v o	ő	\H o	ọ	\d o
o͟	\b o	o͡o	\t oo				

The letters i and j need special treatment with regard to accents, since they should not have their customary dots when accented. The commands `\i` and `\j` produce dot-less i and j as ı and ȷ. Thus to get

> Él está aquí

you must type

```
\’{E}l est\’{a} aqu\’{\i}
```

Some symbols from non-English languages are also available in LaTeX, as shown in the table below:

œ	\oe	Œ	\OE	æ	\ae	Æ	\AE
	\aa		\AA				
ø	\o	Ø	\O	ł	\l	Ł	\L
ß	\ss						
¡	!‘	¿	?‘				

1.2.5. Special symbols

We have see that the input `\LaTeX` produces LaTeX in the output and `\ ` produces a space. Thus TeX uses the symbol \ for a special purpose—to indicate the program that what follows is not text to be typeset but an instruction to be carried out. So what if you want to get \ in your output (improbable as it may be)? The command `\textbackslash` produces \ in the output.

Thus \ is a symbol which has a special meaning for TeX and cannot be produced by direct input. As another example of such a special symbol, see what is obtained from the input below

```
Maybe I have now learnt about 1% of \LaTeX.
```

You only get

> Maybe I have now learnt about 1

What happened to the rest of the line? You see, TeX uses the per cent symbol % as the *comment* character; that is a symbol which tells TeX to consider the text following as ‘comments’ and not as text to be typeset. This is especially useful for a TeX programmer to explain a particularly sticky bit of code to others (and perhaps to himself). Even for ordinary users, this comes in handy, to keep a ‘to do’ list within the document itself for example.

But then, how do you get a percent sign in the output? Just type `\%` as in

```
Maybe I have now learnt about 1\% of \LaTeX.
```

The symbols \ and % are just two of the ten charcaters TEX reserves for its internal use. The complete list is

```
~ # $ % ^ & _ \ { }
```

We have seen how TEX uses two of these symbols (or is it four? Did not we use { } in one of our examples?) The use of others we will see as we proceed.

Also, we have noted that \ is produced in the output by the command \textbackslash and % is produced by \%. What about the other symbols? The table below gives the inputs to produce these symbols.

~	\textasciitilde	&	\&
#	\#	_	_
$	\$	\	\textbackslash
%	\%	{	\{
^	\textasciicircum	}	\}

You can see that except for three, all special symbols are produced by preceding them with a \. Of the exceptional three, we have seen that \~ and \^ are used for producing accents. So what does \\ do? It is used to break lines. For example,

```
This is the first line.\\ This is the second line
```

produces

This is the first line.
This is the second line

We can also give an *optional* argument to \\ to increase the vertical distance between the lines. For example,

```
This is the first line.\\[10pt]
This is the second line
```

gives

This is the first line.

This is the second line

Now there is an extra 10 points of space between the lines (1 point is about $1/72^{nd}$ of an inch).

1.2.6. Text positioning

We have seen that TEX aligns text in its own way, regardless of the way text is formatted in the input file. Now suppose you want to typeset something like this

The TEXnical Institute

Certificate

This is to certify that Mr. N. O. Vice has undergone a course at this institute and is qualified to be a TEXnician.

The Director
The TEXnical Institute

This is produced by

```
\begin{center}
  The \TeX nical Institute\\[.75cm]
    Certificate
\end{center}
\noindent This is to certify that Mr. N. O. Vice has undergone a
course at this institute and is qualified to be a \TeX nician.
\begin{flushright}
  The Director\\
  The \TeX nical Institute
\end{flushright}
```

Here, the commands

```
\begin{center} ... \end{center}
```

typesets the text between them exactly at the center of the page and the commands

```
\begin{flushright} ... \end{flushright}
```

typesets text flush with the right margin. The corresponding commands

```
\begin{flushleft} ... \end{flushleft}
```

places the enclosed text flush with the *left* margin. (Change the `flushright` to `flushleft` and see what happens to the output.)

These examples are an illustration of a LaTeX construct called an *environment*, which is of the form

```
\begin{name} ... \end{name}
```

where *name* is the name of the environment. We have seen an example of an environment at the very beginning of this chapter (though not identified as such), namely the `document` environment.

1.3. FONTS

The actual letters and symbols (collectively called *type*) that LaTeX (or any other typesetting system) produces are characterized by their *style* and *size*. For example, in this book emphasized text is given in *italic* style and the example inputs are given in `typewriter` style. We can also produce smaller and bigger type. A set of types of a particular style and size is called a *font*.

1.3.1. Type style

In LaTeX, a type style is specified by family, series and shape. They are shown in the table 1.1.

Any type style in the output is a combination of these three characteristics. For example, by default we get roman family, medium series, upright shape type style in a LaTeX output. The `\textit` command produces roman family, medium series, italic shape type. Again, the command `\textbf` produces roman family, boldface series, upright shape type.

We can combine these commands to produce a wide variety of type styles. For example, the input

```
\textsf{\textbf{sans serif family, boldface series, upright shape}}
\textrm{\textsl{roman family, medium series, slanted shape}}
```

Table 1.1:

	STYLE	COMMAND
FAMILY	roman	`\textrm{roman}`
	sans serif	`\textsf{sans serif}`
	`typewriter`	`\texttt{typewriter}`
SERIES	medium	`\textmd{medium}`
	boldface	`\textbf{boldface}`
SHAPE	upright	`\textup{upright}`
	italic	`\textit{italic}`
	slanted	`\textsl{slanted}`
	SMALL CAP	`\textsc{small cap}`

produces the output shown below:

sans serif family, boldface series, upright shape
roman family, medium series, slanted shape

Some of these type styles may not be available in your computer. In that case, LaTeX gives a warning message on compilation and substitutes another available type style which it thinks is a close approximation to what you had requested.

We can now tell the whole story of the `\emph` command. We have seen that it usually, that is when we are in the middle of normal (upright) text, it produces italic shape. But if the current type shape is slanted or italic, then it switches to upright shape. Also, it uses the family and series of the current font. Thus

```
\textit{A polygon of three sides is called a \emph{triangle} and a
  polygon of four sides is called a \emph{quadrilateral}}
```

gives

A polygon of three sides is called a triangle *and a polygon of four sides is called a* quadrilateral

while the input

```
\textbf{A polygon of three sides is called a
\emph{triangle} and a polygon of four sides is called a
\emph{quadrilateral}}
```

produces

A polygon of three sides is called a *triangle* and a polygon of four sides is called a *quadrilateral*

Each of these type style changing commands has an alternate form as a *declaration*. For example, instead of `\textbf{boldface}` you can also type `{\bfseries boldface}` to get **boldface**. Note that that not only the name of the command, but its usage also is different. For example, to typeset

By a **triangle**, we mean a polygon of three sides.

if you type

```
By a \bfseries{triangle}, we mean a polygon of three sides.
```

you will end up with

By a **triangle, we mean a polygon of three sides.**

Thus to make the declaration act upon a specific piece of text (and no more), the declaration and the text should be enclosed in braces.

The table below completes the one given earlier, by giving also the declarations to produce type style changes.

	STYLE	COMMAND	DECLARATION
SHAPE	upright	`\textup{upright}`	`{\upshape upright}`
	italic	`\textit{italic}`	`{\itshape italic}`
	slanted	`\textsl{slanted}`	`{\slshape slanted}`
	SMALL CAP	`\textsc{small cap}`	`{\scshape small cap}`
SERIES	medium	`\textmd{medium}`	`{\mdseries medium}`
	boldface	`\textbf{boldface}`	`{\bfseries boldface}`
FAMILY	roman	`\textrm{roman}`	`{\rmfamily roman}`
	sans serif	`\textsf{sans serif}`	`{\sffamily sans serif}`
	typewriter	`\texttt{typewriter}`	`{\ttfamily typewriter}`

These declaration names can also be used as environment names. Thus to typeset a long passage in, say, sans serif, just enclose the passage within the commands `\begin{sffmily} ... \end{sffamily}`.

1.4. TYPE SIZE

Traditionally, type size is measured in (printer) points. The default type that TEX produces is of 10 pt size. There are some *declarations* (ten, to be precise) provided in LATEX for changing the type size. They are given in the following table:

size	`{\tiny size}`	size	`{\large size}`
size	`{\scriptsize size}`	size	`{\Large size}`
size	`{\footnotesize size}`	size	`{\LARGE size}`
size	`{\small size}`	size	`{\huge size}`
size	`{\normalsize size}`	size	`{\Huge size}`

Note that the `\normalsize` corresponds to the size we get by default and the sizes form an ordered sequence with `\tiny` producing the smallest and `\Huge` producing the largest. Unlike the style changing commands, there are no *command-with-one-argument* forms for these declarations.

We can combine style changes with size changes. For example, the "certificate" we typed earlier can now be 'improved' as follows

```
\begin{center}
  {\bfseries\huge The \TeX nical Institute}\\[1cm]
    {\scshape\LARGE Certificate}
```

```
  \end{center}

  \noindent This is to certify that Mr. N. O. Vice has undergone a
  course at this institute and is qualified to be a \TeX nical Expert.

 \begin{flushright}
    {\sffamily The Director\\
    The \TeX nical Institute}
  \end{flushright}
```

and this produces

The TEXnical Institute

CERTIFICATE

This is to certify that Mr. N. O. Vice has undergone a course at this institute and is qualified to be a TEXnical Expert.

The Director
The TEXnical Institute